

tf.compat.v1.Variable Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/Variable

See the [Variables Guide](<https://tensorflow.org/guide/variables>).

Function Signatures

run()

Code Examples

```
tf.compat.v1.Variable(  
    initial_value=None,  
    trainable=None,  
    collections=None,  
    validate_shape=True,  
    caching_device=None,  
    name=None,  
    variable_def=None,  
    dtype=None,  
    expected_shape=None,  
    import_scope=None,  
    constraint=None,  
    use_resource=None,  
    synchronization=tf.VariableSynchronization.AUTO,  
    aggregation=tf.compat.v1.VariableAggregation.NONE,  
    shape=None  
)
```

```
tf.compat.v1.Variable(  
    initial_value=None,  
    trainable=None,  
    collections=None,  
    validate_shape=True,  
    caching_device=None,  
    name=None,  
    variable_def=None,  
    dtype=None,  
    expected_shape=None,  
    import_scope=None,  
    constraint=None,  
    use_resource=None,  
    synchronization=tf.VariableSynchronization.AUTO,  
    aggregation=tf.compat.v1.VariableAggregation.NONE,  
    shape=None  
)
```

```
tf.VariableSynchronization.AUTO
```

```
tf.compat.v1.VariableAggregation.NONE
```

```
import tensorflow as tf
# Create a variable.
w = tf.Variable(, name=)
# Use the variable in the graph like any Tensor.
y = tf.matmul(w, ...another variable or tensor...)
# The overloaded operators are available too.
z = tf.sigmoid(w + y)
# Assign a new value to the variable with `assign()` or a related
method.
w.assign(w + 1.0)
w.assign_add(1.0)
```

```
import tensorflow as tf
# Create a variable.
w = tf.Variable(, name=)
# Use the variable in the graph like any Tensor.
y = tf.matmul(w, ...another variable or tensor...)
# The overloaded operators are available too.
z = tf.sigmoid(w + y)
# Assign a new value to the variable with `assign()` or a related
method.
w.assign(w + 1.0)
w.assign_add(1.0)
```

```
# Launch the graph in a session.
with tf.compat.v1.Session() as sess:
# Run the variable initializer.
sess.run(w.initializer)
# ...you now can run ops that use the value of 'w'...
```

```
# Launch the graph in a session.  
with tf.compat.v1.Session() as sess:  
    # Run the variable initializer.  
    sess.run(w.initializer)  
    # ...you now can run ops that use the value of 'w'...
```

Documentation

See the Variables Guide.

Inherits From: Variable

Used in the notebooks

- [Fitting Dirichlet Process Mixture Model Using Preconditioned Stochastic Gradient Langevin Dynamics](#)

A variable maintains state in the graph across calls to `run()`. You add a variable to the graph by constructing an instance of the class `Variable`.

The `Variable()` constructor requires an initial value for the variable, which can be a `Tensor` of any type and shape. The initial value defines the type and shape of the variable. After construction, the type and shape of the variable are fixed. The value can be changed using one of the assign methods.

If you want to change the shape of a variable later you have to use an assign Op with `validate_shape=False`.

Just like any `Tensor`, variables created with `Variable()` can be used as

inputs for other Ops in the graph. Additionally, all the operators overloaded for the Tensor class are carried over to variables, so you can also add nodes to the graph by just doing arithmetic on variables.

When you launch the graph, variables have to be explicitly initialized before you can run Ops that use their value. You can initialize a variable by running its initializer op, restoring the variable from a save file, or simply running an assign Op that assigns a value to the variable. In fact, the variable initializer op is just an assign Op that assigns the variable's initial value to the variable itself.

The most common initialization pattern is to use the convenience function `global_variables_initializer()` to add an Op to the graph that initializes all the variables. You then run that Op after launching the graph.

If you need to create a variable with an initial value dependent on another variable, use the other variable's `initialized_value()`. This ensures that variables are initialized in the right order.

All variables are automatically collected in the graph where they are created. By default, the constructor adds the new variable to the graph collection `GraphKeys.GLOBAL_VARIABLES`. The convenience function `global_variables()` returns the contents of that collection.

When building a machine learning model it is often convenient to distinguish between variables holding the trainable model parameters and other variables such as a global step variable used to count training steps. To make this easier, the variable constructor supports a `trainable=` parameter. If `True`, the new variable is also added to the graph collection `GraphKeys.TRAINABLE_VARIABLES`. The convenience function `trainable_variables()` returns the contents of this collection. The

various Optimizer classes use this collection as the default list of variables to optimize.

Here, adding `use_resource=True` when constructing the variable will fix any nondeterminism issues:

To use the replacement for variables which does not have these issues:

- Add `use_resource=True` when constructing `tf.Variable`;
- Call `tf.compat.v1.get_variable_scope().set_use_resource(True)` inside a

`tf.compat.v1.variable_scope` before the `tf.compat.v1.get_variable()` call.

Raises

Attributes

Note that this is different from `initialized_value()` which runs the op that initializes the variable before returning its value. This method returns the tensor that is used by the op that initializes the variable.

Child Classes

class `SaveSliceInfo`

Methods

assign

[View source](#)

Assigns a new value to the variable.

This is essentially a shortcut for `assign(self, value)`.

assign_add

[View source](#)

Adds a value to this variable.

This is essentially a shortcut for `assign_add(self, delta)`.

assign_sub

[View source](#)

Subtracts a value from this variable.

This is essentially a shortcut for `assign_sub(self, delta)`.

batch_scatter_update

[View source](#)

Assigns `tf.IndexedSlices` to this variable batch-wise.

Analogous to `batch_gather`. This assumes that this variable and the `sparse_delta` `IndexedSlices` have a series of leading dimensions that are the same for all of them, and the updates are performed on the last dimension of indices. In other words, the dimensions should be the following:

```
num_prefix_dims = sparse_delta.indices.ndims - 1
```

```
batch_dim = num_prefix_dims + 1
```

```
sparse_delta.updates.shape = sparse_delta.indices.shape + var.shape[  
batch_dim:]
```

```
sparse_delta.updates.shape[:num_prefix_dims]  
== sparse_delta.indices.shape[:num_prefix_dims]  
== var.shape[:num_prefix_dims]
```

And the operation performed can be expressed as:

```
var[i_1, ..., i_n,  
sparse_delta.indices[i_1, ..., i_n, j]] = sparse_delta.updates[  
i_1, ..., i_n, j]
```

When `sparse_delta.indices` is a 1D tensor, this operation is equivalent to `scatter_update`.

To avoid this operation one can loop over the first ndims of the variable and using `scatter_update` on the subtensors that result of slicing the first dimension. This is a valid option for `ndims = 1`, but less efficient than this implementation.

count_up_to

[View source](#)

Increments this variable until it reaches limit. (deprecated)

When that Op is run it tries to increment the variable by 1. If incrementing the variable would bring it above limit then the Op raises the exception `OutOfRangeError`.

If no error is raised, the Op outputs the value of the variable before the increment.

This is essentially a shortcut for `count_up_to(self, limit)`.

[View source](#)

In a session, computes and returns the value of this variable.

This is not a graph construction method, it does not add ops to the graph.

This convenience method requires a session where the graph containing this variable has been launched. If no session is passed, the default session is used. See `tf.compat.v1.Session` for more information on launching a graph and on sessions.

experimental_ref

[View source](#)

DEPRECATED FUNCTION

from_proto

[View source](#)

Returns a Variable object created from `variable_def`.

gather_nd

[View source](#)

Gather slices from params into a Tensor with shape specified by indices.

See `tf.gather_nd` for details.

get_shape

[View source](#)

Generated on 10/25/2025 | TensorFlow Documentation Scraper